SOFTWARE ENGINEERING

QUESTION BANK

UNIT - 1

1. Define Software Engineering and explain its need in modern software development.
2. Explain the major components of software with suitable examples.
3. Discuss the key characteristics of software and how they differ from hardware.
4. Compare the Conventional Programming approach with the Software Engineering approach.
5. Explain the importance of process models in Software Engineering.
6. Describe the Waterfall Model with its advantages and limitations.
7. Explain the Prototype Model and its role in requirement clarification.
8. Discuss the Spiral Model with the help of a diagram.
9. Explain Evolutionary Development Models and their advantages.
10. Describe the Iterative Enhancement Model with suitable examples.
11. Explain Rapid Application Development (RAD) and its applicability.
12. What is Software Quality? Explain quality attributes in software.
13. Define Software Quality Assurance (SQA) and explain its objectives.
14. Discuss the role of standards in ensuring software quality.
15. Explain the SEI-CMM model and its maturity levels.
16. Discuss the benefits of adopting CMM in organizations.
17. Explain how quality assurance differs from quality control.
18. Discuss challenges in implementing quality standards.
19. Explain how SQA is integrated into SDLC.
20. Explain continuous improvement in software processes.
21. Describe the responsibilities of an SQA team.

22. Explain the need for SDLC models.

23. Identify the most suitable SDLC model for a banking system and justify.

24. Given a project with unclear requirements and high risk, suggest a suitable SDLC model with reasons.

25. Explain how metrics help in software quality improvement.

26. A defence project involves high risk, complex algorithms, and safety-critical operations**.** Which SDLC model should be adopted? Also Explain how risk analysis is handled in this model.

27. A small team develops software without documentation or formal testing and later faces maintenance issues. Identify problems caused by the conventional approach. Explain how software engineering practices would solve them.

28. A government organization plans to develop a payroll management system with fixed requirements and strict documentation norms. Suggest an appropriate SDLC model. Explain why other models are unsuitable.

29. A software company is developing a banking transaction system where failure may cause financial loss. Identify the best SDLC model. Explain how quality assurance should be integrated.

30. An organization follows SDLC but does not conduct audits or reviews. Identify the risks involved and Suggest corrective SQA measures.

UNIT - 2

1. Define software requirements and explain their importance.
2. Differentiate between Functional and Non-Functional Requirements.
3. Explain User Requirements with examples.
4. Explain the Requirement Engineering Process.
5. Explain requirement elicitation techniques.
6. Explain the role of stakeholders in requirement engineering.
7. Explain requirement analysis and its objectives.
8. Explain traceability in requirements engineering.
9. Explain use cases as a requirement specification technique.
10. What is an SRS document? Explain its purpose.
11. Explain how poor requirements affect project success.
12. Discuss challenges faced during requirement elicitation.
13. What are Interface Requirements? Explain with examples.
14. Write a partial SRS for a Library Management System.
15. Explain the characteristics of a good SRS.
16. Perform feasibility analysis (technical, economic, operational) for an online examination system.
17. Describe the structure of SRS as per IEEE format.
18. A university plans to develop an Online Course Registration System**.** Identify at least five functional and five non-functional requirements. Justify each requirement.
19. A food delivery application must support order tracking, fast response time, and 24×7 availability. Identify user requirements and system requirements.
20. A start-up wants to build a ride-sharing platform with limited funds and a short deadline. Perform a technical, economic, and operational feasibility analysis.
21. A client gives only high-level requirements for a payroll system. Suggest suitable requirement elicitation techniques and justify your choices.
22. A project failed due to frequent requirement changes. Analyze how poor requirement management caused failure and Suggest improvements.

23. A poorly written SRS led to rework and delays. Identify SRS quality issues and suggest improvements.

24. A system meets user requirements but fails acceptance testing. Analyze possible causes related to requirement validation.

25. A government department plans to computerize its legacy record system. Discuss feasibility challenges and solutions.

26. A banking application requires integration with third-party payment gateways. Identify interface requirements and explain their importance.

27. A hospital information system must store patient data securely and allow doctors quick access. Classify the requirements into functional, performance, security, and usability requirements.

28. A development team discovers late that some requirements are not feasible. Explain how better requirement analysis could have prevented this.

29. Write functional and non-functional requirements for an E-commerce Website as per IEEE SRS structure.

30. You are assigned as a requirements engineer for a Smart Campus Management System**.** Identify stakeholders, Perform feasibility analysis, List functional and non-functional requirements, Outline an SRS structure and Explain requirement management strategy.

UNIT - 3

1. Explain basic concepts of software design.
2. Compare Top-down and Bottom-up approaches.
3. Differentiate between software errors, faults, and failures.
4. Explain verification and validation with the help of an example.
5. Describe Top-down design approach with examples.
6. Explain the relationship between coupling and cohesion with the help of suitable example.
7. Define coupling and explain its types.
8. Explain cyclomatic complexity and its significance with the help of an example.
9. Define software design and explain its importance.
10. Explain testing for non-functional requirements.
11. Compare white-box and black-box testing techniques.
12. Describe integration testing strategies.
13. Design test plan and test cases for an online banking login module.
14. Explain Bottom-up design approach with examples.
15. A team is developing a Library Management System and decides to decompose the system into major modules such as Issue Book, Return Book, and Fine Calculation. Identify the design approach used. Draw a structure chart and explain data and control coupling.
16. Given a Student Information System, decompose the system using a top-down approach and justify module boundaries.
17. Define cohesion and explain its types.
18. A team redesigns an existing monolithic system into independent modules. Explain how structure charts help improve design clarity.
19. A software system frequently breaks when a single module is modified. Identify the likely coupling problem. Suggest design improvements.
20. A module performs input validation, database updates, and report generation. Identify the type of cohesion and Suggest refactoring.

21. Explain how increasing cohesion and reducing coupling can improve testability in a payroll system.

22. A system crashes only under heavy load. Identify possible faults and testing strategies to detect them.

23. Explain verification and validation activities for an Online Voting System.

24. Design equivalence class test cases for a login module with username and password constraints.

25. Apply boundary value analysis for an ATM withdrawal limit feature.

26. Explain how code inspection can detect defects early in a safety-critical system.

27. Design a structure chart and test cases for an Online Food Ordering System.

28. Consider the following program segment:

```
void check(int x)
{
    if (x > 0)
        printf("Positive");
    else
        printf("Non-positive");
}
```

Draw the control flow graph (CFG) and compute the cyclomatic complexity.

29. Identify valid and invalid equivalence classes for a password field with length 8–16 characters.

30. For a system with three input fields, determine the minimum number of black-box test cases.

31. Given a program with 20 edges and 16 nodes, determine Cyclomatic complexity Number of independent paths.

32. Develop a design and testing strategy for a Hospital Management System, covering:

- Design approach
- Coupling & cohesion
- Testing levels
- White-box and black-box testing techniques

UNIT - 4

1. Define software maintenance and explain its importance.
2. Explain different types of software maintenance.
3. Describe corrective maintenance with the help of an example.
4. Discuss challenges faced during software maintenance.
5. Describe the Constructive Cost Model (COCOMO).
6. Explain project planning activities.
7. Explain configuration identification and establishment.
8. Discuss advantages and limitations of COCOMO.
9. Explain role of scheduling management in projects.
10. An organization modifies its payroll software to comply with new government tax rules. Identify the type of maintenance involved and justify your answer.
11. A banking application frequently crashes due to previously undetected bugs after deployment. Identify the maintenance type and suggest preventive measures.
12. A company upgrades its software to support a new operating system and hardware platform. Identify the maintenance type and explain challenges involved.
13. A product team plans maintenance activities before release. Explain the role of preventive maintenance.
14. Frequent minor fixes in a deployed system increase maintenance cost significantly. Analyze reasons and suggest solutions.
15. A large project involves multiple teams and dependencies. Explain how effective project planning can reduce risks.
16. A software project of 32 KLOC is categorized as an Organic project. Estimate the effort and development time using the Basic COCOMO model.
17. Two projects of the same size use different development modes. Analyze the difference in effort estimation using COCOMO.
18. A software team loses track of multiple versions during development. Explain how version control helps prevent this problem.
19. A software project suffers from poor estimation, missed deadlines, and frequent changes. Analyze issues related to cost estimation, scheduling, and configuration management.

20. A company wants to reduce maintenance cost over the product life cycle. Suggest strategies involving maintenance planning and configuration management.

21. You are the project manager for an Online Banking System. Identify maintenance types, Estimate cost using COCOMO, Plan schedule using CPM/PERT and Describe configuration management activities.

22. A software project consists of the following activities:

Activity	Predecessor	Duration (Days)
A	--	4
B	A	6
C	A	5
D	B	7
E	C	4
F	D, E	3

Draw the project network diagram, Identify the critical path and Calculate the minimum project completion time.

23. The following table gives time estimates (in weeks) for activities in a software project:

Activity	Optimistic (t_o)	Most Likely (t_m)	Pessimistic (t_p)
A	2	4	8
B	3	6	9
C	1	2	3
D	4	5	10

Calculate the expected time for each activity, Compute the variance of each activity and Identify the most critical activity.

24. A project has the following activity details:

Activity	Predecessor	Optimistic (t_o)	Most Likely (t_m)	Pessimistic (t_p)
A	--	2	3	4
B	A	3	5	9
C	A	2	4	6
D	B	4	6	8
E	C	3	5	7
F	D, E	2	3	4

Calculate expected time and variance, Draw the project network, Identify the critical path and Compute expected project duration.

25. A legacy banking system requires frequent bug fixes and occasional feature enhancements. Analyze how different types of maintenance contribute to overall lifecycle cost and suggest strategies to reduce maintenance effort.

26. A project is delayed despite adequate manpower. Analyze possible planning and tracking failures and suggest corrective measures.

27. A project manager uses expert judgment instead of algorithmic models for cost estimation. Analyze the advantages and limitations of this approach compared to COCOMO.

28. Discuss the limitations of the Basic COCOMO model and analyze situations where Intermediate COCOMO is more suitable.

29. A software project suffers from frequent requirement changes, poor cost estimation, and uncontrolled versions. Analyze how integrated project management and configuration management can address these issues.

30. A company plans to develop an Online Examination System. Activities, dependencies, and time estimates are provided. Management wants to know:

- Expected completion time
- Critical path
- Probability of meeting the deadline

Apply PERT for time estimation, Identify the critical path and Analyze schedule risk.

UNIT - 5

1. Define Agile methodology and explain its need.
2. Compare Agile and traditional SDLC models.
3. Explain responsibilities of Product Owner.
4. Explain Scrum framework.
5. Explain the principles of the Agile Manifesto.
6. Describe burndown chart and its significance.
7. Explain benefits of Agile methodologies.
8. Explain the role of manager in Scrum.
9. Compare different scaled Agile frameworks.
10. Explain Agile implementation using a real-world example.
11. A software company follows a rigid Waterfall model and faces frequent requirement changes from customers. Analyze how adopting Agile principles can address these challenges. Map the problems to relevant principles of the Agile Manifesto.
12. An organization claims to be Agile but avoids customer collaboration and frequent deliveries. Analyze whether the organization is truly Agile and justify your reasoning.
13. Describe the role of Scrum Master.
14. Describe sprint planning process.
15. A Scrum team frequently misses sprint goals due to unclear backlog items. Analyze the role of Product Backlog refinement and Product Owner responsibilities.
16. A Scrum team experiences conflicts between the Scrum Master and Project Manager. Analyze role clarity issues and explain correct Scrum role responsibilities.
17. A product backlog grows continuously without prioritization. Analyze how this affects sprint planning and delivery predictability.
18. A team does not conduct sprint retrospectives. Analyze the long-term impact on team performance and product quality.
19. During sprint execution, multiple user stories remain incomplete. Analyze the impact on sprint backlog and sprint commitment.
20. Management demands detailed upfront plans in an Agile project. Analyze how Agile metrics can still provide visibility and control.

21. A project requires frequent releases, strict coding standards, and continuous testing. Analyze why Extreme Programming (XP) may be suitable.

22. An organization wants to eliminate waste and improve flow efficiency. Analyze the suitability of Lean principles in software development.

23. Compare Scrum and Kanban for a customer support software system with unpredictable workload.

24. An organization plans to scale Scrum across departments while maintaining autonomy. Analyze the suitability of Scrum@Scale.

25. An organization uses different Agile practices across teams leading to inconsistency. Analyze how Disciplined Agile can help standardize decision-making.

26. You are appointed as an Agile Coach in a mid-sized organization. Analyze steps required for successful Agile adoption.

27. A distributed Agile team faces communication issues. Analyze how Agile ceremonies and tools can mitigate these challenges.

28. Compare traditional project management and Agile management using a real-world software project.

29. A company wants faster delivery without compromising quality. Analyze how Scrum practices ensure both speed and quality.

30. Design an Agile–Scrum strategy for an Online Food Delivery System, covering:

- Agile principles
- Scrum roles
- Product & sprint backlog
- Sprint ceremonies
- Tracking using burndown chart
- Scaling approach